# SoundHound CIE Interview Prep

ROAST Deep Dive

Resume Critic AI — Multi-Agent Pipeline

June 2026

# ROAST — Resume Critic AI (Deep Dive)

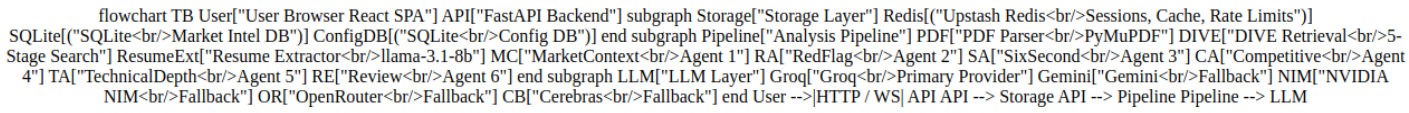
> **For**: Complete refresher. Understand everything you built. **Goal**: Answer any interview question about ROAST architecture, agents, pipeline, or tradeoffs.

## What is ROAST?

A web app where users upload a resume → get a market-calibrated AI review. 6 agents analyze the resume in parallel, grounded in real market data.

**User flow**: Upload PDF → pick role/company/market → wait 2-3 min → get detailed review with TL;DR, market pulse, red flags, competitive position, action plan.

## High-Level Architecture




## Tech Stack

| Component | Technology | Why Chosen |
|---|---|---|
| Backend | FastAPI | Async, native WebSocket, auto OpenAPI docs |
| Frontend | React 19 + Vite 8 + Tailwind v4 | Modern, fast HMR |
| LLM Primary | Groq (multi-key) | 14,400 RPD for 8B models, free tier |
| LLM Fallbacks | Gemini, NIM, OpenRouter, Cerebras | 5 providers = no single point of failure |
| Cache/Session | Upstash Redis | Serverless Redis, free tier, survives restarts |
| Market DB | SQLite (market_intel.db) | Pre-built via monthly ingestion. No external DB needed |
| Config DB | SQLite (market_config.db) | Company lists, salary bands, role weights |
| PDF | PyMuPDF (fitz) | Fast, extracts links from annotation layer |
| Embeddings | Gemini gemini-embedding-001 | 3072-dim. API-based, no GPU needed |
| Search | SQLite FTS5 + numpy | BM25 + vector. No external vector DB needed |
| Observability | Langfuse v4 | LLM call tracing, user feedback |
| Deployment | Docker → Cloud Run | Single container, auto-scaling to 0 |

## Project Structure

```
roast/
├── backend/
│   ├── main.py              # FastAPI entry, routers, startup/shutdown
│   ├── config.py            # Env vars: API keys, rate limits, thresholds
│   ├── market_data.py       # 982 lines. SQLite config DB CRUD + seed data
│   ├── pdf_reader.py        # Resume PDF → text + link extraction
│   ├── agents/              # 6 AI agents
│   │   ├── resume_extractor.py    # Pre-pipeline: structured resume facts
│   │   ├── market_context_agent.py # Agent 1 (runs first): weight calibration
│   │   ├── red_flag_agent.py      # Agent 2: 11 red flag categories
│   │   ├── six_second_agent.py    # Agent 3: recruiter scan + trajectory
│   │   ├── competitive_agent.py   # Agent 4: percentile + CTC estimation
│   │   ├── technical_depth_agent.py # Agent 5: project eval with web search
│   │   ├── review_agent.py        # Agent 6 (runs last): full review synthesis
│   │   ├── followup_agent.py      # Post-review: follow-up Q&A
│   │   └── prompts/               # System prompts per agent
│   ├── llm/                 # Provider clients + routing
│   │   ├── router.py              # Routes to providers with fallback chains
│   │   ├── groq_client.py         # Distributed key rotation via Redis
│   │   ├── gemini_client.py       # Gemini fallback
│   │   ├── openrouter_client.py   # Emergency fallback
│   │   ├── nvidia_nim_client.py   # NVIDIA NIM fallback
```

```
│   │   ├── cerebras_client.py     # Cerebras fallback
│   │   ├── langfuse_client.py     # Observability wrapper
│   │   └── circuit_breaker.py     # 3-state per-provider breaker
│   ├── pipeline/
│   │   └── orchestrator.py  # 480 lines. Coordinates ALL agents
│   ├── retrieval/
│   │   └── dive.py          # 645 lines. 5-stage retrieval pipeline
│   ├── routes/
│   │   ├── analyse.py       # POST /api/analyse
│   │   ├── session.py       # POST /api/session-init
│   │   ├── websocket.py     # WS /api/ws/{session_id}
│   │   ├── followup.py      # POST /api/followup
│   │   ├── cron.py          # POST /refresh-market-intel
│   │   └── token_feedback.py # POST /api/token, /api/feedback
│   ├── storage/
│   │   ├── session_store.py # Redis session CRUD
│   │   ├── rate_limit.py    # IP-based, IST midnight reset
│   │   └── redis_client.py  # Upstash Redis singleton
│   └── corpus/              # Anonymized signal DB for calibration
├── frontend/                # React SPA
│   └── src/
│       ├── App.jsx          # View routing: landing vs analysis
│       ├── lib/api.js       # 8 API functions + WebSocket creator
│       ├── hooks/
│       │   ├── useWebSocket.js    # WS with polling fallback + heartbeat
│       │   └── useInferenceToggle.js # localStorage toggle
│       └── components/
│           ├── LandingPage.jsx    # Upload form (440 lines)
│           ├── AnalysisProgress.jsx # 8-step progress (201 lines)
│           ├── ResultsPage.jsx    # Results orchestrator (229 lines)
│           ├── TLDRBlock.jsx      # Shortlist chance + fixes
│           ├── MarketPulse.jsx    # Salary, sentiment, signals
│           ├── ReviewDocument.jsx # 6-section accordion (266 lines)
│           ├── Feedback.jsx       # Thumbs + token unlock
│           └── SkeletonLoader.jsx
├── ingestion/               # Monthly market intel builder
│   ├── pipeline.py          # Orchestrator. 10 queries/combo
│   ├── extractor.py         # LLM signal classification
│   ├── search.py            # SQLite FTS5 CRUD
│   ├── embeddings.py        # Gemini embedding + vector search
│   ├── tavily_client.py     # Budget-tracked Tavily search
│   └── breaking_signal.py   # 24h breaking news layer
├── tests/                   # 7 test files
└── scripts/
    ├── prepopulate.py       # Ingestion for all 70 combos
    └── reembed.py           # Regenerate missing embeddings
```

---

## Agent Pipeline

sequenceDiagram participant U as User participant API as FastAPI participant DIVE as DIVE Retrieval participant RE as ResumeExtractor participant MCA as MarketContextAgent participant PARA as 4 Parallel Agents participant RVA as ReviewAgent participant LLM as LLM Providers U->>API: POST /api/analyse (PDF + metadata) API->>DIVE: Search market intelligence API->>RE: Extract resume facts DIVE-->>API: Market signals RE-->>API: Structured resume Note over API: Pipeline orchestrator starts API->>MCA: Step 1: Calibrate weights MCA->>LLM: llama-3.1-8b LLM-->>MCA: weight_map MCA-->>API: Market calibration par Step 2: Parallel Agents API->>PARA: RedFlagAgent API->>PARA: SixSecondAgent API->>PARA: CompetitiveAgent API->>PARA: TechnicalDepthAgent end PARA-->>API: All 4 results collected API->>RVA: Step 3: Synthesize review RVA->>LLM: llama-3.3-70b (with 5-level fallback) LLM-->>RVA: Full review Note over RVA: Quality gate: 250-2000 words, inference chains, action plan RVA-->>API: Final review API-->>U: WebSocket: section_complete events API-->>U: WebSocket: complete event

## The 6 Agents in Detail

### Agent 1: MarketContextAgent (runs FIRST)

**What it does**: Takes DIVE market signals + job description → produces a **weight_map** that tells every other agent what to focus on.

```
Example output for "SDE at Indian Startup":
{
  "dsa_weight": 0.30,
  "projects_weight": 0.25,
  "cgpa_weight": 0.15,
  "experience_weight": 0.20,
  "college_tier_weight": 0.10
}
```

**Why it runs alone**: Every other agent needs these weights. RedFlagAgent needs to know "is 7.5 CGPA concerning?" CompetitiveAgent needs weights to compute percentile.

**Key logic**: After LLM produces weights, `_enforce_weight_map()` cross-checks against DB rules (role overrides, company overrides, experience defaults). LLM can suggest, but DB rules are enforced.

**Fallback**: If LLM fails, returns DB defaults + LOW confidence. System still works.

### Agent 2: RedFlagAgent (parallel)

**What it does**: Hunts 11 categories of resume problems.

**11 categories**:

1. **Hedge words** ("familiar with," "exposed to") — shallow knowledge
2. **Unverifiable skills** — claims without evidence
3. **Missing contact** — no email/phone/LinkedIn
4. **CGPA consequences** — below role threshold
5. **Buried lead** — impressive fact in boring sentence
6. **Responsibility without outcome** — just listed duties
7. **Date arithmetic** — gaps/overlaps in timeline
8. **Hidden CGPA** — graduation year shown but GPA hidden
9. **Generic filler** — anyone could write this
10. **ATS keyword gaps** — missing terms for target role
11. **Role-specific mistakes** — wrong tech stack claims

**Quality gate**: Each flag needs: location_in_resume ≥ 10 chars, fix ≥ 20 chars, inference_chain ≥ 50 chars. Generic phrases like "could be improved" are rejected.

### Agent 3: SixSecondAgent (parallel)

**What it does**: Simulates a recruiter's F-pattern scan (eyes scan top → left → diagonal in 6 seconds). Also analyzes career trajectory.

**Input**: First 200 words (for scan) + full resume (for trajectory).

**Output**: First-impression score + career narrative + trajectory assessment.

### Agent 4: CompetitiveAgent (parallel)

**What it does**: Estimates where the candidate ranks in the applicant pool.

**Outputs**:

- Percentile range (e.g., "45th–55th percentile")
- Expected CTC range (e.g., "₹12-18 LPA")
- Strengths and weaknesses for THIS specific role

**Calibration**: If corpus has ≥30 similar profiles → "calibrated" (based on real data). If <30 → "based on market data" (uses salary bands from market_config.db).

### Agent 5: TechnicalDepthAgent (parallel, 90s timeout)

**What it does**: Evaluates projects with genuine technical understanding.

**Unique capability**: Agentic loop. It calls DuckDuckGo search to verify project claims. "Claims to have built a transformer from scratch" → searches "transformer from scratch tutorial" → determines if it's tutorial-level or genuinely novel.

**Difficulty levels**: Tutorial / Intermediate / Advanced / Exceptional (role-calibrated).

**Skip filter**: Known tech (100+ terms) bypasses web search to save time.

### Agent 6: ReviewAgent (runs LAST)

**What it does**: Synthesizes ALL upstream outputs into a flowing English review.

**5-section review**:

1. What's Working
2. What's Hurting You
3. Career Story
4. Competitive Position
5. Action Plan

**Quality gate**: Word count 250-2000, inference chains present, specific follow-up questions, action plan has clear steps.

**Fallback chain (5 levels)**: llama-3.3-70b → gpt-oss-20b → qwen3-32b → gemini-2.5-flash-lite → NIM → OpenRouter.

**Final fallback**: `_assemble_partial_review()` — deterministic Python synthesis. No LLM needed. Lower quality but NEVER fails.

```
def _assemble_partial_review(six_second, red_flags, competitive, market_context):
    high_flags = [f for f in red_flags.red_flags if f.severity == "HIGH"]
    flag_text = " ".join([f.flag for f in high_flags[:3]]) if high_flags else "No critical issues found."

    return ReviewOutput(
        tldr_shortlist_chance=competitive.percentile_estimate.range,
        tldr_biggest_blocker=flag_text,
        tldr_fix_first=competitive.highest_leverage_change,
        confidence="LOW",
        whats_working_section=" ".join(competitive.strengths_vs_pool[:2]),
        whats_hurting_section=" ".join([f.inference_chain for f in high_flags[:2]]),
        career_story_section=six_second.career_story,
        competitive_position_section=competitive.percentile_estimate.reasoning,
        action_plan_section=competitive.highest_leverage_change,
        jd_alignment_section="",
        six_second_followups=["What can I improve about my first impression?"],
    )
```

**Trigger condition**: Called only after 2 LLM attempts with 2 different providers (with retry instructions) all fail. Copies upstream outputs directly — no LLM involvement.

---

## Orchestrator Deep Dive (`pipeline/orchestrator.py`)

The orchestrator is 480 lines. It's the brain of the entire analysis pipeline.

### Execution Order: 3 Stages, 6 Agents

```
Stage 1 (sequential): JD Parser → DIVE Retrieval → ResumeExtractor
Stage 2 (sequential): MarketContextAgent (runs alone, needs DIVE output)
Stage 3 (parallel):   ┌── RedFlagAgent
                      ├── SixSecondAgent
                      ├── CompetitiveAgent
                      └── TechnicalDepthAgent
Stage 4 (sequential): ReviewAgent (with fallback chain + quality gate)
Stage 5 (async):      Corpus store + bullet curation (fire-and-forget)
```

### Semaphore System — 4 Levels of Concurrency

```
_groq_sem = asyncio.Semaphore(2)        # Max 2 concurrent Groq calls
_gemini_sem = asyncio.Semaphore(1)      # Max 1 concurrent Gemini call
_global_sem = asyncio.Semaphore(3)      # Max 3 simultaneous full pipelines
_tech_depth_sem = asyncio.Semaphore(3)  # gpt-oss-120b: 24K TPM with 3 keys
```

Each semaphore prevents resource exhaustion at a different level:

- **Groq sem (2)**: Multiple API keys share a pool-wide RPD. Limiting concurrency prevents daily quota burn.
- **Global sem (3)**: Prevents 100 concurrent users from overwhelming free-tier providers.
- **Tech depth sem (3)**: gpt-oss-120b has 24K TPM across 3 keys. Semaphore keeps bursts under limit.

### Error Propagation — Never Crash

Every parallel agent is wrapped with `return_exceptions=True`:

```
red_flags, six_second, competitive, tech_depth = await asyncio.gather(
    _run_red_flag(session, request, market_context, progress_callback),
    _run_six_second(session, request, resume_facts, progress_callback),
    _run_competitive(session, request, resume_facts, progress_callback),
    _run_tech_depth(session, request, resume_facts, progress_callback),
    return_exceptions=True
)
```

Each failure is caught and replaced with a fallback output:

```
if isinstance(red_flags, Exception):
    red_flags = RedFlagOutput(red_flags=[], visual_scan_notes="", confidence="LOW")
```

**The pipeline NEVER crashes.** Every agent has a guaranteed fallback schema.

### Confidence Override

```
if full_market_ctx.raw_signal_count >= 10 and market_context.confidence == "LOW":
    market_context.confidence = "HIGH"  # LLM is too conservative
```

When DIVE retrieves 10+ signals but the MarketContextAgent still says "LOW" confidence, the orchestrator overrides it. The LLM is inherently conservative; the orchestrator knows better.

### Prompt Versioning for A/B Testing

```
def _compute_prompt_hash(session, request):
    all_prompts = "...".join([p1, p2, p3, p4, p5])
    return hashlib.sha256(all_prompts.encode()).hexdigest()[:8]

def _get_prompt_variant(session_id):
    return "A" if int(hashlib.md5(session_id.encode()).hexdigest()[-1], 16) % 2 == 0 else "B"
```

Every analysis records which prompt variant was used. Enables A/B testing prompt changes without separate deployments.

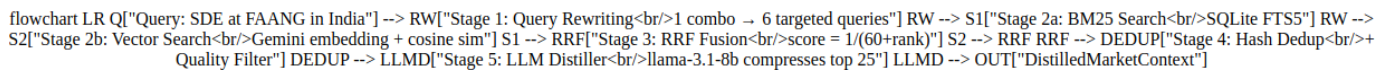
### Section-Level Streaming

```
async def _emit_section(session_id, section_name, result):
    await ws_manager.emit(session_id, {
        "event": "section_complete",
        "data": {"section": section_name, "result": result}
    })
    # Also persist to Redis for reconnection recovery
    redis.setex(f"session:{session_id}:{section_name}", 3600, json.dumps(result))
```

Each completed section is emitted via WebSocket AND persisted in Redis. If the user disconnects and reconnects, `_get_completed_sections()` replays completed sections.

---

## DIVE Retrieval (5 Stages)




### Stage 1 — Query Rewriting

One combo → 6 targeted queries:

```
def _build_retrieval_queries(role, company_type, market, experience_level):
    return [
        f"{role} hiring sentiment {company_type} {market}",
        f"{role} required skills tools {company_type} {market}",
        f"{role} competitive pool applicants {market}",
        f"{role} definition expectations {experience_level} {market}",
        f"{role} red flags resume {company_type} {market}",
        f"{role} salary format norms {market}",
    ]
```

Each query targets a different aspect (salary, skills, hiring trends, company news, competition, market sentiment).

### Stage 2 — Parallel Search

BM25 and vector search run simultaneously:

```
async def _parallel_search(queries, combo_id):
    bm25_results, vector_results = await asyncio.gather(
        asyncio.to_thread(_bm25_search, queries),     # SQLite FTS5
        asyncio.to_thread(_vector_search, queries),   # Numpy cosine similarity
    )
```

- **BM25**: SQLite FTS5 — runs all 6 queries against `market_signals` FTS5 index, deduplicates by signal `id`
- **Vector**: Gemini embedding (3072-dim) → numpy cosine similarity against stored BLOBs. Combined query string embedded once.

### Stage 3 — RRF Fusion

Standard Reciprocal Rank Fusion:

```
RRF_K = 60
def _rrf_fusion(bm25_results, vector_results):
    scores = {}
    for rank, row in enumerate(bm25_results, start=1):
        scores[row["id"]] = scores.get(row["id"], 0) + 1 / (RRF_K + rank)
    for rank, row in enumerate(vector_results, start=1):
        scores[row["id"]] = scores.get(row["id"], 0) + 1 / (RRF_K + rank)
    sorted_ids = sorted(scores.keys(), key=lambda x: scores[x], reverse=True)
    return [rows_by_id[i] for i in sorted_ids]
```

Documents appearing in BOTH BM25 and vector results get double the score — they float to the top.

### Stage 4 — Hash Dedup + Quality Filter

```
def _hash_dedup(results, limit=25):
    seen_hashes = set()
    filtered = []
    for row in results:
        content_hash = hashlib.md5(row["content"][:200].encode()).hexdigest()
        if content_hash not in seen_hashes:
            seen_hashes.add(content_hash)
            if len(row.get("content", "")) >= 30:  # quality filter
                filtered.append(row)
    return filtered[:25]
```

### Stage 5 — LLM Distiller

llama-3.1-8b compresses top 25 signals into a concise `DistilledMarketContext` for MarketContextAgent:

```
DISTILLER_SYSTEM = """You are a market intelligence analyst. Given raw hiring signals,
extract: key salary ranges, in-demand skills, hiring sentiment, common red flags.
Output: concise JSON with fields: salary_range, top_skills, sentiment, key_insights, signal_count"""
```

On failure: returns DB defaults with `confidence="LOW"`.

### Caching Strategy (3-tier)

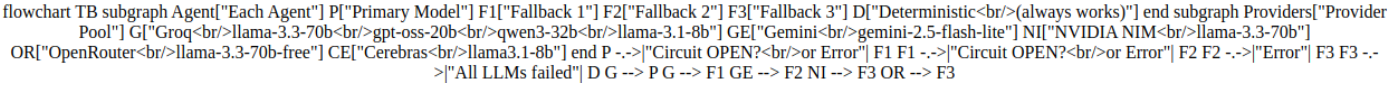
| Cache | Key Pattern | TTL | Purpose |
|---|---|---|---|
| **Snapshot** | `snapshot:{role}:{company}:{market}` | 15 days | Full DIVE result |
| **Previous snapshot** | `snapshot_prev:{role}:{company}:{market}` | 60 days | For diff comparison |
| **Breaking signal** | `breaking:{market}:{category}:{company}` | 24 hours | Fresh news layer |

### Warmup

On server startup, `warmup_cache()` refreshes top 15 combos (by historical `combo_count:` in Redis). Each stale (>24h) combo gets a fresh DIVE run. This ensures the first user of the day doesn't wait for retrieval.

---

## LLM Routing & Fallback




**Circuit breaker per provider**:

- CLOSED: Normal operation
- OPEN (after 3 failures): Fast-fail for 5 minutes
- HALF-OPEN (after 5 min): Allow 1 test call
- Success → CLOSED. Failure → OPEN again.

**Provider fallback chains per agent**:

| Agent | Primary | Fallback 1 | Fallback 2 | Fallback 3 |
|---|---|---|---|---|
| Review | llama-3.3-70b (Groq) | gpt-oss-20b | qwen3-32b | Gemini → NIM → OpenRouter |
| RedFlag | llama-3.3-70b (Groq) | llama-3.1-8b | — | — |
| SixSecond | qwen3-32b (Groq) | llama-3.1-8b | — | — |
| Competitive | gpt-oss-20b (Groq) | NIM | — | — |
| TechnicalDepth | gpt-oss-120b (Groq) | llama-3.1-8b | — | — |
| MarketContext | llama-3.1-8b (Groq) | — | — | — |

---

## Circuit Breaker Implementation (`llm/circuit_breaker.py`)

### 3-State Machine

```
class CircuitBreaker:
    def __init__(self, name, failure_threshold=3, cooldown_seconds=300):
        self.name = name
        self.state = "closed"        # closed → open → half_open
        self.failures = 0
```

```
        self.threshold = 3           # 3 consecutive failures → open
        self.cooldown = 300           # 5 minutes cooldown
        self.last_failure_time = 0

    def record_failure(self):
        self.failures += 1
        if self.failures >= self.threshold:
            self.state = "open"
            self.last_failure_time = time.time()

    def record_success(self):
        if self.state == "half_open":
            self.state = "closed"
            self.failures = 0

    def should_skip(self):
        if self.state == "open":
            if time.time() - self.last_failure_time > self.cooldown:
                self.state = "half_open"
                return False  # Allow 1 probe call
            return True  # Fast-fail
        return False  # Let the call through
```

| State | Behavior |
|---|---|
| **CLOSED** | Normal operation. Calls go through. |
| **OPEN** | Fast-fail for 5 minutes. No calls attempted. |
| **HALF-OPEN** | Allow 1 test call. Success → CLOSED. Failure → OPEN again. |

### Key Details

- **In-memory only** — NOT persisted to Redis. Restart resets all circuits.
- **Not distributed** — each worker/container has its own circuit breaker state.
- **Time-based recovery** — transitions HALF-OPEN after exactly 300s, not after a configurable interval.
- **Single success closes** — one successful call in HALF-OPEN resets to CLOSED immediately.

### 5 Singleton Breakers

```
# Module-level singletons — one per provider
groq_cb = CircuitBreaker("groq")
gemini_cb = CircuitBreaker("gemini")
cerebras_cb = CircuitBreaker("cerebras")
openrouter_cb = CircuitBreaker("openrouter")
nvidia_nim_cb = CircuitBreaker("nvidia_nim")
```

---

## Groq Key Rotation (`llm/groq_client.py`)

### Distributed Round-Robin via Redis

```
_keys = [k.strip() for k in GROQ_API_KEYS.split(",")]

async def _get_key_index():
    idx = await redis.incr("groq:round_robin_counter")
    return (idx - 1) % len(_keys)
```

Every API call atomically increments `groq:round_robin_counter` in Redis. The index is computed as `(counter - 1) % key_count`. This distributes evenly across all workers/containers since the counter lives in Redis.

### RPD Tracking Per Model Per Key

```
RPD_LIMITS = {
    "llama-3.1-8b-instant":    14400,
    "llama-3.3-70b-versatile":  1000,
    "qwen/qwen3-32b":           1000,
    "openai/gpt-oss-20b":       1000,
    "openai/gpt-oss-120b":      1000,
}
```

Each model × each key has a counter at `groq:rpd:{model}:{key_index}`. TTL set with 0-300s random jitter to expire at midnight UTC (prevents thundering herd).

### 429 Handling

```
except RateLimitError:
    key_idx = _rotate(key_idx)  # (current + 1) % len(_keys)
    client = AsyncGroq(api_key=_keys[key_idx])
    await asyncio.sleep(backoff[attempt])  # [2, 4, 8]
```

On 429: rotate to next key, retry with exponential backoff (2s, 4s, 8s).

### Proactive Fallback Warnings

RPM remaining is extracted from response headers and stored in Redis. If < 50 remaining, logged as warning for operator alerting.

### Qwen3 Thinking Mode Handling

```
if "</think>" in text:
    text = text[text.index("</think>") + len("</think>"):].strip()
elif text.startswith("<think>"):
    raise RuntimeError("qwen3_thinking_truncated")  # triggers retry with higher temp
```

Qwen3 models wrap their reasoning in `<think>...</think>` tags. The client strips them. If the thinking is truncated (no closing tag), it retries with higher temperature.

### Langfuse Tracing

Fire-and-forget trace on every successful LLM call (if session_id + agent_name provided). Tracks: model, provider, prompt tokens, completion tokens, latency, and agent name.

---

## Rate Limiter (`storage/rate_limit.py`)

### IP-Based, 3 Analyses/Day

```
FREE_ANALYSES_PER_DAY = 3
IST = ZoneInfo("Asia/Kolkata")

def check_and_increment(ip):
    key = f"ratelimit:{ip}"
    count = redis.incr(key)
    if count == 1:
        ttl = _seconds_until_midnight_ist()
        redis.expire(key, ttl)
    allowed = count <= 3
    if not allowed:
        redis.decr(key)  # undo the atomic incr
    return {"allowed": allowed, "count": count, "remaining": max(0, 3 - count), "limit": 3}
```

### Midnight Reset (IST)

```
def _seconds_until_midnight_ist():
    now = datetime.now(IST)
    midnight = datetime.combine(now.date(), time(0, 0, 0), tzinfo=IST)
    if midnight <= now:
        midnight += timedelta(days=1)
    return int((midnight - now).total_seconds())
```

### Token Unlock Override

In `analyse.py`: if rate limited AND `redis.get(f"token_unlocked:{session_id}")` exists, the rate limit is skipped and the token key is deleted (one-time use). Users can get an email token for an extra analysis.

### Anti-Bot Timing Gate

```
if elapsed < 1.0:  # Reject requests faster than 1 second (likely a bot)
    raise HTTPException(429)
```

---

## Session Store (`storage/session_store.py`)

### Redis Key Pattern

```
session:{session_id}  # TTL: 3600s (1 hour)
```

### Fields Stored

```
session = {
    "session_id": str(uuid.uuid4()),
    "role": str,                    # e.g. "SDE1"
    "market": str,                  # e.g. "India"
    "company_type": str,            # e.g. "Indian Product Company"
    "experience_level": str,        # e.g. "Junior"
    "created_at": int(time.time()),
    "status": "pending"             # pending → processing → completed / failed
}
```

After pipeline starts (`analyse.py`), additional fields are added: `resume_text`, `resume_links`, `page_count`, canonicalized `company_type`.

### Update Pattern

```
def update_session(sid, updates):
    data = redis.get(f"session:{sid}")       # Get current
    data.update(updates)                      # Merge
    redis.setex(f"session:{sid}", 3600, data) # Set with TTL refresh
```

### Section-Level State (for Reconnection Recovery)

Each pipeline stage stores its output at:

```
session:{sid}:MarketContext
session:{sid}:RedFlag
session:{sid}:SixSecond
session:{sid}:Competitive
session:{sid}:TechnicalDepth
session:{sid}:Review
```

On WebSocket reconnect, `_get_completed_sections(sid)` reads all 6 keys and re-emits `section_complete` events for completed ones.

---

## Analyse Route Flow (`routes/analyse.py`)

### Multipart Upload → Pipeline Trigger

```
POST /api/analyse (multipart/form-data)
│
├── 1. Validate session exists (GET from Redis)
│
├── 2. Idempotency check — reject if status=processing or completed
│
├── 3. Anti-bot gate — reject if form filled in < 1.0s
│
├── 4. Rate limit check — IP-based, 3/day IST
│   └── Token unlock override for 4th analysis
│
├── 5. Extract PDF
│   ├── Content-type validation
│   ├── PyMuPDF extract text + links
│   ├── Validate: max 5MB, max 3 pages, min 200 chars, max 15K chars
│   ├── Temp file → extract → unlink immediately
│   └── track in _temp_files set for crash cleanup
│
├── 6. Update session → "processing"
│
├── 7. BackgroundTasks.add_task(_run_pipeline_and_stream, ...)
│   └── HTTP response returns immediately
│
└── 8. Return {session_id, status: "processing", pages: N, chars: N}
```

### Background Pipeline Task

```
@router.post("/api/analyse")
async def analyse(...):
    ...
    background_tasks.add_task(_run_pipeline_and_stream, session_id, ...)
    return {"session_id": session_id, "status": "processing", ...}
```

FastAPI's `BackgroundTasks` ensures the HTTP response returns immediately while the pipeline runs asynchronously in the background. The **only** way to track progress is the WebSocket.

### IP Detection

```
ip = request.headers.get("x-forwarded-for", request.client.host or "unknown")
# x-forwarded-for handles reverse proxies (Cloud Run)
```

### Temp File Cleanup

```
_temp_files: set[str] = set()
atexit.register(_cleanup_temp_files)  # Clean up on crash
```

Every temp PDF file is registered for cleanup if the process crashes mid-extraction.

---

## WebSocket Handler Deep Dive (`routes/websocket.py`)

### Message Protocol

```
Server → Client: {"event": "section_complete", "data": {"section": str, "result": dict}}
Server → Client: {"event": "ping"}
Client → Server: "pong"
```

### Heartbeat Loop

```
async def heartbeat_loop(session_id, ws, interval=10):
    while session_id in _connections:
        await asyncio.sleep(10)
        try:
            await ws.send_text(json.dumps({"event": "ping"}))
        except WebSocketDisconnect:
            break
```

30-second receive timeout. If no message in 30s, checks if session is completed/failed and breaks.

### Reconnection Recovery

On WebSocket connect:

1. Read all 6 section keys from Redis (`session:{sid}:*`)
2. For each completed section, re-emit `section_complete` event
3. If all 6 complete, emit `complete` event
4. If status is `failed`, emit `error` event

### Disconnect Handling

```
except WebSocketDisconnect:
    pass
finally:
    heartbeat_task.cancel()
    _connections.pop(session_id, None)
```

Stale connections cleaned up by `cleanup_stale_connections()` after 5 minutes of inactivity.

### Share Preview

`GET /share/{session_id}` returns TL;DR block only (no resume text, no red flags). Cached 7 days in Redis.

---

## Prompt Engineering Patterns

### Prompt Architecture

All prompts go through `build_system_prompt()` in `agents/prompts/template.py`:

1. **Base persona**: `"You are an expert resume analyst specialising in {role} at {company_type} in {market}"`
2. **Market calibration** from DIVE (injected separately as context)
3. **Role calibration** from `market_config.db` (role weights, company lists)
4. **Company section**: Top 8 companies for this role+type+market (dynamically queried)
5. **Agent-specific task**: The actual instruction for THIS agent
6. **Universal constraints** (6 rules): generic advice ban, prompt injection resistance, JSON-only, edge cases
7. **Token budget warning**: 3000 token limit for responses
8. **Agent-specific constraints**: e.g., "do not contradict facts from earlier sections"

### Notable Patterns

**Inference Chain Enforcement** ( `review_prompt.py` ):

```
"Recruiter sees [exact observation] → assumes [specific assumption] → decides [concrete outcome]"
```

Quality gate checks for `→` arrow characters. Missing → triggers retry with specific instruction.

**Anti-Generic Guardrails** ( `red_flag_prompt.py` ):

```
BANNED_PHRASES = [
    "recruiters look for", "is important to", "this shows that",
    "lacks quantifiable", "should include metrics",
    "demonstrates that you", "will negatively impact"
]
```

Flags with >1 banned phrase are rejected at parse time.

**Role-Specific Calibration** ( `review_prompt.py:108-171` ):

- Fresher: judge projects/CGPA
- Junior: judge tech depth
- Mid: judge architectural decisions
- Senior: judge org impact and leadership

**Company-Type-Aware Personas** ( `review_prompt.py:8-105` ): Separate hiring manager personas for: service companies, FAANG, startups, MNC GCCs, semiconductor, consulting — each with specific city references (Whitefield for GCCs, Koramangala for startups).

**Anti-Hallucination Rules** ( `review_prompt.py:247-253` ):

```
- ALWAYS read the resume text before making any claim
- NEVER say 'no mention of X' before searching
- Every weakness claim must be VERIFIABLE against the resume text
```

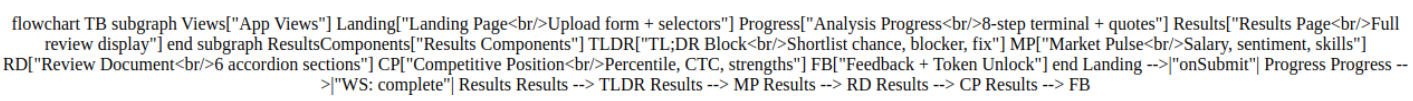
---

## 16 API Endpoints

| Method | Route | What It Does |
|---|---|---|
| POST | `/api/session-init` | Create session (role, company, market, experience) |
| POST | `/api/analyse` | Upload PDF resume (multipart) |
| WS | `/api/ws/{session_id}` | Real-time progress + results stream |
| GET | `/api/session/{id}/state` | Poll for reconnection after WS drop |
| GET | `/api/session/{id}` | Raw session data |
| POST | `/api/followup` | One-click follow-up Q&A |
| POST | `/api/feedback` | Useful/not-useful vote |
| POST | `/api/token` | Request email token for extra analysis |
| POST | `/api/token/verify` | Validate token |
| POST | `/refresh-market-intel` | QStash monthly cron (HMAC verified) |
| GET | `/health` | Liveness + total analysis count |
| GET | `/share/{session_id}` | Public TL;DR preview |

---

## Frontend Architecture




**Two views only** (manual `useState` switching — no router library):

1. **Landing**: Form with 4 selectors (role, experience, company, market), PDF dropzone (5MB max), optional context/JD/GitHub, consent checkboxes
2. **Analysis** → either Progress (streaming) or Results (complete)

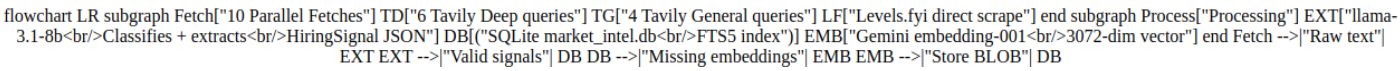
**useWebSocket hook** (110 lines):

- Connects to `WS /api/ws/{session_id}`

- Processes: `ping` → reply `pong`, `section_complete` → accumulate results, `complete` → show results
- **Polling fallback**: If WS disconnects, poll `GET /api/session/{id}/state` every 5s
- **Heartbeat**: 3 missed pings (45s) → switch to polling

---

## Ingestion Pipeline (Monthly)




**What it does**: Every month, for each of ~70 role/company/market combos:

1. Fire 10 search queries (6 deep + 4 general) via `asyncio.gather`
2. Handle truncated results via Jina Reader fallback
3. Classify each result with llama-3.1-8b (is this a job posting? salary survey? blog? discard?)
4. Extract structured HiringSignal: signal_type, skills, salary_range, sentiment, trust_weight
5. Store in SQLite `market_signals` table with auto-synced FTS5 index
6. Generate Gemini embeddings (3072-dim), store as BLOB

**Optimization story**: Original was sequential — 70 combos took 10+ hours. After refactoring to async parallel with semaphore (3 concurrent), same work takes <1 hour. The lesson: start concurrent from day one.

---

## Key Architecture Decisions

1. **SQLite instead of vector DB**: A full Pinecone/Qdrant deployment was overkill. SQLite FTS5 + numpy cosine similarity handles the scale (~10K signals). No external service = zero cost, zero ops.
2. **5-provider fallback**: Free-tier LLM APIs are unreliable. Circuit breakers + fallback chains ensure no single provider failure blocks a user. The deterministic fallback guarantees something useful is always returned.
3. **Agent parallelism**: Agents 2-5 run in parallel (asyncio.gather). This cuts total analysis time from ~3 min to ~1.5 min. Agent 1 must run first (its output is needed by everyone). Agent 6 must run last.
4. **WebSocket + polling dual-mode**: WebSocket for real-time streaming during normal operation. HTTP polling fallback for reconnection. Heartbeat monitor detects silent disconnects.
5. **$0 infrastructure**: All services on free tiers (Groq, Gemini, Upstash, Tavily, Resend, Cloud Run free quota). This constrains architecture (rate limits, budget tracking) but keeps operating cost at zero.